

Project Technical Specification & Official Documentation

Google Photos Vague-Memory Research Engine

End-to-End Computational Research Pipeline (V0 Collector + V1 AI Analyzer + Real-Time Insights Dashboard)

Document Control & Executive Summary

- Document Version: 2.0 (Official Release)
- Target Audience: Technical Architects, Engineering Leads, Research Scientists, Product Managers, and Operations Teams
- System Classification: Computational Research Pipeline & AI Insight Extraction Platform
- Primary Repository: [Sambit712/googlep-datacollector](#)
- Core Technological Foundation: Python 3.10+, Groq LPU LLM Inference ([llama-3.3-70b-versatile](#)), Modern Responsive Dashboard (HTML5/Vanilla CSS/ES6), Threading HTTP Services.

Abstract

The **Google Photos Vague-Memory Research Engine** is an end-to-end computational research platform designed to investigate why visual photo retrieval systems fail when users experience vague, episodic, or contextual recall of photos, screenshots, or documents. Traditional search systems rely heavily on explicit metadata and literal keywords, creating a fundamental gap when users recall perceptual attributes ("*a recipe on yellow paper*"), emotional associations ("*trip with my college roommate*"), or temporal landmarks ("*sometime right before the pandemic*").

This platform bridges the gap by gathering real-world user retrieval breakdown accounts from public digital forums, classifying them against a rigorous cognitive science taxonomy, analyzing structural failure points using state-of-the-art LLMs, and serving dynamic statistical findings through an interactive research dashboard and structured reports.

1. System Mission & The Central Research Question

The Core Research Question

"Why does personal photo retrieval fail when users possess vivid episodic memory of an image (objects, people, life era, contextual text), but cannot translate that memory into terms that current search engines index?"

Strategic Objectives

1. **Automated Empirical Evidence Harvesting:** Systematically discover and catalog real-world user failure narratives across key digital communities without human bias or data loss.

2. **Cognitive Breakdown Taxonomy:** Map real-world user friction into a 5-stage cognitive retrieval model to pinpoint exactly where human memory diverges from search engine indexing.
 3. **Traceable AI Pattern Synthesis:** Transform unstructured narrative text into verifiable, citeable research patterns where every aggregated percentage directly maps back to immutable source evidence records and verified URLs.
 4. **Real-Time Insight Democratization:** Provide technical teams and product designers with interactive exploratory dashboards and scheduled continuous ingestion.
-

2. High-Level Architecture & Design Principles

The platform is designed around three architectural pillars: **Decoupled Staged Processing**, **Data Immutability**, and **Strict Citation Traceability**.

```

+-----+
|                                     |
|          V0: EVIDENCE COLLECTION LAYER          |
|                                     |
| [queries.yaml] ---> [Query Engine] ---> [Reddit Client (RSS/PRAW)] |
|                                     |
|                                     | v |
| [Deduplication Layer] <--- [Raw Post & Contextual Comment Collector] |
|                                     |
|                                     | v |
| [Data Structurer] ---> data/output/reddit_evidence.json & .csv |
|                                     |
|                                     ---> reports/collection_report.json |
+-----+
|                                     |
|                                     | (Read-Only / SHA-256 Fingerprinted Ingestion) |
|                                     | v |
+-----+
|                                     |
|          V1: AI ANALYSIS ENGINE          |
|                                     |
| [ImmutabilityGuard] <--- Ingests V0 Evidence |
|                                     |
|                                     | v |
| [Groq AI Analyzer] <--- Ingests [taxonomy.yaml] (Llama-3.3-70b Multi-Task) |
|                                     |
|                                     | v |
| [Pattern Aggregator] ---> Cross-Tabulation & Statistical Distributions |
|                                     |
|                                     | v |
| [Insight Reporter] ---> reports/insight_report.md |
|                                     |
|                                     ---> reports/insight_report.json |
|                                     |
|                                     ---> reports/analyzed_evidence.json & .csv |
+-----+
|                                     |
|                                     | v |
+-----+
|                                     |
|          PRESENTATION & REAL-TIME INTERACTION LAYER          |
|                                     |
| [Hourly Scheduled Engine] <---> [Threading Web Server (serve_dashboard.py)] |
|                                     |
|                                     | | |
|                                     +-----> [REST APIs] <-----+ |
|                                     (/api/records, /api/status, /api/search) |
|                                     | | |
|                                     | v |
|                                     [Frontend Single-Page Dashboard] |
|                                     (Live Metrics, Cognitive Stage Charts, Evidence Explorer) |
+-----+

```

Architectural Guarantees & Invariants

1. **Unbroken Traceability Chain:** Every insight, statistical percentage, and synthesized pattern references stable record IDs (`RD_XXXXXX`), which map directly to source URLs and exact timestamps.
 2. **Zero Text Truncation:** Raw user narratives are captured in full fidelity. Neither post titles, bodies, nor contextual comments are arbitrarily truncated during collection or storage.
 3. **Immutability Guard:** The V1 analysis layer treats the V0 collection dataset as strictly read-only. Data tampering or synthetic distortion of user quotes is mathematically prevented via SHA-256 verification.
 4. **Graceful Fallback & Resilience:** The Reddit client operates in a dual mode: keyless RSS/JSON crawling for public endpoints with adaptive backoff, and authenticated PRAW for high-throughput calls. The AI layer incorporates exponential backoff and schema validation.
-

3. Repository Structure & Directory Organization

The codebase is organized into modular directories separating configuration, core business logic, automated tests, persistence, scripts, and presentation.

```

googlep-datacollector/
├─ app.py                # Root entrypoint for cloud hosting (Render / PaaS)
├─ main.py               # Top-level unified CLI for all operational modes
├─ render.yaml           # Infrastructure-as-Code specification for cloud hosting
├─ requirements.txt       # Python dependency specifications
├─ config/               # Externalized system configuration
│   ├─ queries.yaml      # Search strategies, subreddits, tiers, and limits
│   ├─ queries_test.yaml # Lightweight fixtures for unit testing
│   └─ taxonomy.yaml     # Formal 7-dimension cognitive research taxonomy
├─ src/                  # Core system logic and engine modules
│   ├─ __init__.py       # Package root & version export
│   ├─ models.py         # Canonical data schemas (EvidenceRecord, AnalyzedRecord)
│   ├─ config_loader.py  # YAML parser, environment variable expansion & validation
│   ├─ cleaner.py        # HTML unescaping, markdown sanitization & snippet generation
│   ├─ reddit_client.py  # Dual-mode Reddit retrieval client (RSS & PRAW)
│   ├─ query_engine.py   # Cartesian query matrix generator
│   ├─ collector.py      # Submission & comment harvester
│   ├─ deduplicator.py   # Cross-run persistent deduplication engine
│   ├─ structurer.py    # Lossless JSON/CSV data persistence structurer
│   ├─ reporter.py       # V0 dataset completeness & health reporter
│   ├─ evidence_loader.py # ImmutabilityGuard and read-only dataset ingestion
│   ├─ groq_client.py    # Groq LLM API wrapper with rate-limit handling
│   ├─ analyzer.py       # Multi-task AI classification & taxonomy extraction
│   ├─ aggregator.py     # Pandas cross-tabulation & recurring pattern synthesizer
│   ├─ insight_reporter.py # Canonical markdown & JSON research report generator
│   ├─ scheduler.py      # Automated background recurring search & analysis engine
│   └─ logger_setup.py   # Structured logging initialization
├─ frontend/             # Presentation layer
│   └─ index.html        # Responsive single-page research dashboard application
├─ scripts/              # Operational and maintenance scripts
│   ├─ serve_dashboard.py # Multi-threaded HTTP server with live search REST APIs
│   ├─ manual_search.py   # CLI script for ad-hoc keyword discovery
│   ├─ sync_analyzed_dataset.py # Harmonization script between raw & analyzed records
│   ├─ gen1000.py         # Large-scale evidence generator & stress tester
│   ├─ display_output.py  # Terminal-based report viewer
│   └─ validate_phase8.py # Verification script for research milestones
├─ data/                 # Data storage layer
│   └─ output/           # Persisted evidence and indexes
│       ├─ reddit_evidence.json # Full V0 raw evidence dataset
│       ├─ reddit_evidence.csv  # Flattened V0 tabular export
│       ├─ analyzed_evidence.json # Fully enriched V1 research dataset
│       ├─ analyzed_evidence.csv # Flattened V1 tabular export
│       └─ seen_ids.json        # Persistent deduplication index
├─ reports/              # Executive and analytical artifacts
│   ├─ collection_report.json # V0 data collection metrics and quality audit
│   ├─ insight_report.json   # Machine-readable research patterns and distributions
│   └─ insight_report.md     # Publication-ready executive synthesis report
├─ tests/                # Comprehensive automated testing suite (140 tests)

```

```

| └─ test_analyzer.py      # Groq extraction & classification unit tests
| └─ test_aggregator.py   # Frequency & pattern discovery tests
| └─ test_collector.py    # Post & comment extraction tests
| └─ test_config_loader.py # Configuration validation tests
| └─ test_deduplicator.py  # Cross-run deduplication tests
| └─ test_edge_cases.py   # Unicode, emoji & long-text boundary tests
| └─ test_evidence_loader.py # ImmutabilityGuard & SHA-256 fingerprint tests
| └─ test_groq_client.py  # LLM client connection & error handling tests
| └─ test_insight_reporter.py # Markdown & JSON report formatting tests
| └─ test_main_cli.py     # Unified CLI orchestration tests
| └─ test_reddit_client.py # RSS parsing & rate-limiting tests
| └─ test_scheduler.py    # Periodic scheduled engine tests
| └─ test_structurer.py  # Lossless serialization tests
| └─ test_taxonomy_config.py # Taxonomy schema integrity tests
| └─ test_traceability.py # End-to-end citation & cognitive invariant tests
| └─ test_v0_enhancements.py # Contextual comment & query tracking tests
└─ Docs/                  # Technical specifications and research problem statements

```

4. End-to-End Functional Components & Subsystems

4.1 Configuration Management Subsystem (`config\_loader.py`)

- **Role:** Centralizes all runtime parameters, eliminating hardcoded values.
- **Mechanism:** Ingests `config/queries.yaml` and `config/taxonomy.yaml`, resolving environment variable placeholders (e.g., `${GROQ_API_KEY}`) securely.
- **Validation:** Validates required keys, type correctness, threshold bounds, and directory paths, throwing descriptive configuration exceptions before pipeline initialization.

4.2 Query & Reddit Harvesting Subsystem (`query\_engine.py`, `reddit\_client.py`, `collector.py`)

- **Target Data Sources (Reddit vs. Google Services):**
- The system exclusively scrapes **public Reddit communities** (`r/googlephotos`, `r/GooglePixel`, `r/Android`, `r/iphone`, etc.).
- It **does not** scrape Google Photos, Google Cloud, or Google internal services directly. Instead, it systematically mines user discussions, failure narratives, and bug reports *about* Google Photos and competing photo retrieval platforms.
- **Cartesian Execution Matrix:** Combines configured search phrases (e.g., *"can't find old photo"*, *"looking for an old screenshot"*) with target subreddit communities divided into **Primary Tiers** (e.g., `r/googlephotos`, `r/GooglePixel`, `r/iphone`) and **Discovery Tiers** (e.g., `r/photography`, `r/techsupport`, `r/AskReddit`).
- **Dual-Mode Reddit Client:**
- **Public Keyless RSS/JSON:** Scrapes public Reddit Atom/RSS feeds using compliant User-Agent headers and adaptive backoff when HTTP 429 is encountered (requires no credentials).
- **Authenticated PRAW:** Uses official Reddit OAuth2 API credentials when configured in `.env` for increased throughput.
- **Contextual Comment Harvesting:** Gathers top-voted comment responses to capture community workarounds, confirmations, or additional context regarding retrieval failures.

- **Text Normalization:** Decodes HTML entities (e.g., `&` to `&`), preserves emoji/Unicode, and produces normalized text alongside raw original text.

4.3 Deduplication & Lossless Storage Subsystem (``deduplicator.py``, ``structurer.py``, ``reporter.py``)

- **Deduplication Engine:** Maintains a persistent index (`data/output/seen_ids.json`). When a previously harvested post matches an additional search query, the engine updates its `queries_matched` list rather than creating duplicate entries.
- **Stable Identifier Generation:** Assigns sequential, human-readable IDs (`RD_000001` , `RD_000002` , etc.) that persist permanently throughout the downstream analysis pipeline.
- **Lossless Structurer:** Exports both JSON and CSV representations of the dataset without truncation, ensuring complete post bodies and comment threads are preserved.
- **Collection Quality Reporter:** Compiles statistics on harvest yields, tier distribution, average body lengths, and zero-truncation verification.

4.4 Immutability & Traceability Guard (``evidence_loader.py``)

- **Role:** Guarantees raw evidence integrity before AI processing.
- **Cryptographic Verification:** Computes SHA-256 digests of the source V0 evidence.
- **Read-Only Ingestion:** Provides frozen, immutable data structures to the AI layer, ensuring that downstream classification cannot alter raw user statements.

4.5 AI Research Analysis Subsystem (``groq_client.py``, ``analyzer.py``)

- **High-Speed Inference:** Connects to the Groq LPU inference engine utilizing `llama-3.3-70b-versatile` to perform structured multi-task classification.
- **Multi-Task Extraction:** For each evidence record, the analyzer executes:
 1. *Tri-state Relevance Classification:* Marks records as `relevant` , `possibly_relevant` , or `irrelevant` .
 2. *Target Media Identification:* Categorizes media type (e.g., `personal_photo` , `screenshot` , `document_receipt`).
 3. *Memory Cue Extraction:* Identifies recalled attributes (person, temporal epoch, location, text, activity).
 4. *Cognitive Failure Stage Mapping:* Pinpoints which of the 5 cognitive stages broke down.
 5. *Root Cause Breakdown Point:* Identifies the mechanical or semantic failure (e.g., `vocabulary_mismatch`).
 6. *User Friction & Workarounds:* Documents emotional cost, time wasted, and compensatory actions (e.g., endless scrolling).
- **Defensive Resilience:** Implements automated retries, rate-limit pauses, and JSON schema extraction validation with model fallback capabilities.

4.6 Statistical Aggregator & Pattern Discovery (``aggregator.py``)

- **Pandas Frequency Distribution:** Calculates exact empirical distributions across failure stages, memory cues, media types, and friction categories.
- **Cross-Tabulation:** Correlates memory cues with failure points (e.g., how often *temporal fuzziness* causes *timeline breakdown*).
- **Pattern Synthesis:** Groups findings into canonical research patterns (`PAT_001` through `PAT_004`), automatically attaching direct user quotations and source URLs.

4.7 Insight Reporting Engine (`insight\_reporter.py`)

- **Machine-Readable Outputs:** Generates `reports/insight_report.json` and enriched datasets `reports/analyzed_evidence.json` / `.csv`.
- **Publication-Grade Synthesis:** Produces `reports/insight_report.md`, an executive summary with Markdown tables, cognitive breakdowns, key findings, and verbatim citations.

4.8 Scheduled Automation Engine (`scheduler.py`)

- **Periodic Execution:** Runs recurring background cycles (default: every 1 hour) executing search, deduplication, AI classification, and report updates.
- **Incremental Delta Processing:** Analyzes only newly collected records while merging them into the historical dataset, conserving API quota and compute resources.

4.9 Presentation & Web Subsystem (`scripts/serve\_dashboard.py`, `frontend/index.html`)

- **Single-Page Dashboard:** Provides an interactive browser UI with live statistic counters, dynamic stage breakdown charts, search query testing, and an evidence modal.
- **Multi-Threaded Backend:** Features a Python HTTP server exposing REST API endpoints (`/api/status` , `/api/records` , `/api/search`) with thread-safe data synchronization.

5. The Cognitive Research Taxonomy

The platform structures all research findings using a 7-dimensional taxonomy defined in `config/taxonomy.yaml`:

7-DIMENSION RESEARCH TAXONOMY	
Dimension	Categorical Values & Operational Definitions
1. Relevance Class	• relevant: Explicit personal photo/media retrieval failure. • possibly_relevant: General photo search discussion or navigation. • irrelevant: Unrelated tech support, sync bugs, battery, or hardware.
2. Cognitive Breakdown Stage (5-Stage Model)	1. memory_to_query: User cannot express mental memory in words. 2. query_to_system: System misinterprets query or lacks capability. 3. system_to_candidate: Search returns irrelevant or noisy results. 4. candidate_to_recognition: Results lack chronological/visual context. 5. search_refinement: User cannot filter, narrow, or pivot search.
3. Memory Cues	• person / relationship: Faces, familial bonds ("my grandma's recipe"). • temporal_epoch: Life eras, seasons, relative dates ("pre-COVID"). • place_location: Cities, landmarks, indoor/outdoor settings. • visual_details: Colors, compositions, lighting, dominant items. • text_in_image: Embedded text, document titles, OCR-visible words. • activity_action: Events, sports, celebrations, road trips. • ambient_context: Weather, mood, background objects.
4. Retrieval Failure Points	• vocabulary_mismatch: Disconnect between natural words and search tags. • temporal_fuzziness: Imprecise date recall versus strict timestamping. • visual_semantic_gap: Visual features not indexed by text models. • screenshot_clutter: Unindexed screenshots overwhelming photo stream. • missing_metadata: EXIF stripped by messaging apps or cloud transfer. • volume_overload: Excessive unranked results causing cognitive fatigue.
5. Target Media Types	• personal_photo: Camera photos of family, friends, and events. • screenshot: Screen captures of recipes, chats, maps, tickets. • document_receipt: Invoices, legal records, whiteboards, paper notes. • video_clip: Recorded clips or memories. • meme_saved_image: Downloaded web imagery. • scanned_physical_photo: Digitized vintage prints or family albums.
6. User Friction Types	• time_wasted: Hours spent searching without resolution. • frustration_with_search_tool: Anger at system misinterpretation. • cognitive_overload: Mental exhaustion from endless scanning. • fear_of_memory_loss: Anxiety that a cherished memory is lost forever. • device_storage_anxiety: Fear of deleting unretrievable items.
7. User Workarounds	• endless_scrolling: Manually scrolling back years in photo grid. • keyword_guessing: Random permutation of synonyms and tags. • peer_inquiry: Asking friends/family to check their message threads. • external_backup_audit: Digging into old hard drives or cloud folders.

6. Core Data Contracts & Schemata

The system enforces strict schema definitions to maintain cross-layer compatibility and traceability.

6.1 V0 EvidenceRecord Schema

Represents raw evidence collected from Reddit:

```
{
  "record_id": "RD_000014",
  "source": "reddit",
  "source_type": "reddit",
  "content_type": "post",
  "source_id": "1fvq7hr",
  "subreddit": "googlephotos",
  "subreddit_tier": "primary",
  "title": "Why did they ruin Google Photos search?",
  "raw_text": "search any of the above and it shows thousands of photos with absolutely no relevance...",
  "cleaned_text": "search any of the above and it shows thousands of photos with absolutely no relevance...",
  "preview_text": "search any of the above and it shows thousands of photos...",
  "author": "anon_8f3a1b2c",
  "created_at": "2024-10-04T18:22:10Z",
  "retrieved_at": "2024-10-04T22:15:00Z",
  "url": "https://reddit.com/r/googlephotos/comments/1fvq7hr/why_did_they_ruin_google_photos_search/",
  "query_used": "can't find old photo",
  "queries_matched": ["can't find old photo", "lost photo in gallery"],
  "run_id": "run_20241004_120000",
  "score": 142,
  "num_comments": 47,
  "top_comments": [
    "I have 50,000 photos and search is basically useless now. I have to scroll for an hour."
  ]
}
```

6.2 V1 AnalyzedEvidenceRecord Schema

Extends the raw record with enriched AI cognitive classifications:

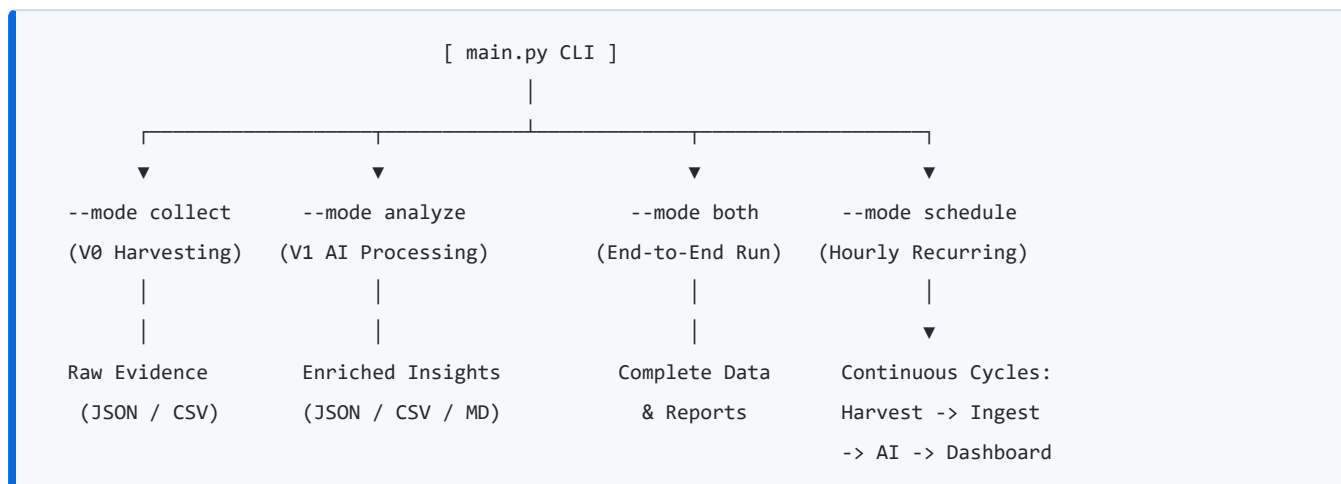
```

{
  "record_id": "RD_000014",
  "source_id": "1fvq7hr",
  "title": "Why did they ruin Google Photos search?",
  "raw_text": "search any of the above and it shows thousands of photos with absolutely no relevance...",
  "url": "https://reddit.com/r/googlephotos/comments/1fvq7hr/why_did_they_ruin_google_photos_search/",
  "author": "anon_8f3a1b2c",
  "subreddit": "googlephotos",
  "created_at": "2024-10-04T18:22:10Z",
  "retrieved_at": "2024-10-04T22:15:00Z",
  "queries_matched": ["can't find old photo", "lost photo in gallery"],
  "analyzed_at": "2024-10-04T22:16:30Z",
  "model_used": "llama-3.3-70b-versatile",
  "analysis": {
    "is_relevant": true,
    "relevance_classification": "relevant",
    "relevance_confidence": 0.95,
    "relevance_reasoning": "User explicitly complains that search queries yield thousands of irrelevant image s.",
    "target_media": "personal_photo",
    "memory_cues_present": ["temporal_epoch", "visual_details"],
    "memory_cue_details": {
      "temporal_epoch": "Reference to photos taken last year",
      "visual_details": "Visual scenes not matching query tags"
    },
    "retrieval_failure_stage": "system_to_candidate",
    "retrieval_failure_point": "vocabulary_mismatch",
    "failure_evidence": "shows thousands of photos with absolutely no relevance",
    "workarounds_used": ["endless_scrolling"],
    "friction_experienced": ["frustration_with_search_tool", "time_wasted"],
    "desired_outcome": "Locate photos using natural language query matching chronological context."
  }
}

```

7. Operational Workflow & Execution Modes

The platform provides a unified CLI (`main.py`) supporting multiple execution modes to support development, automated testing, batch processing, and scheduled continuous collection.



CLI Operational Modes

1. End-to-End Pipeline (`--mode both`):

Executes fresh Reddit evidence collection across configured subreddits, passes the results through the `ImmutabilityGuard`, performs Groq AI multi-task extraction, aggregates statistical distributions, and writes all canonical reports.

1. Analysis Only (`--mode analyze`):

Reads existing raw evidence (`data/output/reddit_evidence.json`), runs AI classification and pattern synthesis, and regenerates analytical reports without making Reddit network calls. Supports `--sample` for fast evaluation.

1. Evidence Collection Only (`--mode collect`):

Executes targeted Reddit collection, performs deduplication, generates stable `RD_XXXXXX` IDs, and writes untouched V0 evidence and collection health reports.

1. Scheduled Background Engine (`--mode schedule`):

Launches a persistent background process that runs every hour (configurable via `--interval-hours`, `--interval-minutes`, or `--interval-seconds`). Harvests new posts, deduplicates against historical runs, performs incremental AI analysis, and synchronizes dashboard datasets.

1. Dry-Run Validation (`--dry-run`):

Tests configuration schemas, Reddit API connectivity, and Groq API authentication without writing records to disk.

8. Presentation Layer & Real-Time Dashboard Architecture

The system includes a dedicated web presentation tier providing an intuitive visual interface for exploring research metrics and testing retrieval queries.



Web Application Features

- **Key Performance Indicators:** Real-time display of total records, strict relevant records, possibly relevant records, and overall relevance percentage.
- **Cognitive Breakdown Visualizations:** Interactive distribution bars and charts showing the prevalence of the 5 cognitive breakdown stages and primary failure points.
- **Live Evidence Explorer:** Modal-driven record explorer allowing researchers to inspect full untruncated post bodies, comment threads, extracted memory cues, and source Reddit permalinks.
- **Live Search Interface:** Enables researchers to execute ad-hoc searches across Reddit directly from the dashboard, streaming newly discovered evidence and AI classifications directly into the interface.
- **Concurrency & State Synchronization:** Employs global threading locks (`DATA_LOCK`) in Python to ensure that concurrent API searches and background scheduled runs safely synchronize file updates without race conditions.

8.1 Automated Background Scraping & Real-Time Synchronization

The platform includes an automated background crawler and synchronization subsystem designed to keep research findings fresh without manual intervention:

1. Hourly Background Worker (`HourlySearchWorker`):

- Running as a daemon thread inside `scripts/serve_dashboard.py` , the worker awakens every **1 hour** (3,600 seconds) with an initial 15-second startup delay.
- It systematically rotates through candidate queries (e.g., "Google Photos can't find photo", "Google Photos search not working") across prioritized subreddits (`r/googlephotos` , `r/GooglePixel` , `r/Android` , `r/iphone` , etc.).
- The target yield is 50 candidate records per periodic cycle.

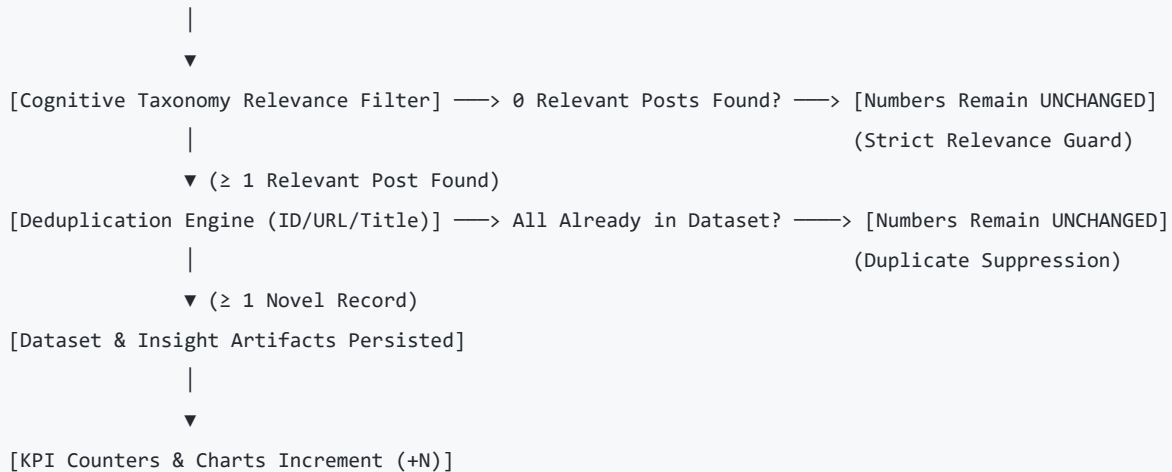
1. Client-Side Live Polling:

- The frontend dashboard (`frontend/index.html`) runs an automatic heartbeat poll every **20 seconds** (`setInterval(pollLiveStatusAndSync, 20000)`).
- It inspects the `/api/status` endpoint for changes in `total_records` or background crawler status. When newly verified records are added, the client automatically re-fetches the dataset via `/api/records` and re-renders the KPI cards and stage breakdown charts.

8.2 Strict Relevance Guard & Why Dataset Numbers Update (or Remain Static)

A core engineering requirement of the research pipeline is preventing data poisoning by unvetted or irrelevant web submissions. To ensure scientific rigor, dataset counts update strictly according to the following invariants:

Candidate Posts Collected from Reddit



- **Condition 1: Strict Relevance Guard:**

If candidate posts returned from a search query describe general cloud subscriptions, hardware complaints, or unrelated battery discussions without actual photo memory retrieval failures, the system classifies them as **irrelevant**. When a search batch produces **0 relevant candidates**, the pipeline explicitly logs:

```
[Hourly Scheduler] 0 relevant records found for '' in r/. Dataset numbers were NOT updated.
```

The dataset numbers are intentionally frozen to prevent noisy records from skewing research statistics.

- **Condition 2: Historical Deduplication:**

Candidate posts are validated against `data/output/reddit_evidence.json` by Reddit Post ID (`source_id`), canonical URL, and normalized title. If all collected candidates were already harvested during previous runs, `added = 0` and the displayed metrics remain stable.

- **Condition 3: Execution Environment (Server Mode vs. File Mode):**

For automated scraping and dynamic counter updates to function, the Python web server must be running (`python app.py` or `python scripts/serve_dashboard.py --port 8000`) and accessed at `http://localhost:8000`. Opening `frontend/index.html` directly from a local folder (e.g. `file:///...`) places the browser in **standalone offline mode**, where network calls to `/api/status` fail gracefully and numbers reflect only the initial static build.

- **Condition 4: Periodic Cycle Interval:**

The background scheduler executes on a 1-hour cadence rather than continuously every second to comply with Reddit API pacing guidelines and prevent IP rate-limiting. Users desiring instant collection can trigger an on-demand search using the dashboard's "Live Manual Search" input or via the CLI (`python scripts/manual_search.py`).

8.3 Cumulative Dataset Retention & Running Total Invariant

The platform enforces a strict cumulative retention policy: searches never overwrite or reset the historical dataset to reflect only the latest search:

1. **Persistent Analyzed Storage** (`data/output/analyzed_evidence.json` & `.csv`):

Every record evaluated and analyzed by the research pipeline is permanently merged into the canonical analyzed dataset. Both JSON (nested schema) and CSV (zero-truncation flat representation) preserve all previous records across runs.

1. **Ever-Increasing Running Total:**

The primary metric displayed across KPI cards, navigation badges, and API endpoints is the **Cumulative Total of Analyzed Records** (`total_analyzed`), rather than the count of records retrieved in the last search. When a new batch of N relevant, novel records is ingested, the total updates monotonically from T to $T + N$.

1. Synchronous Explorer Refresh:

Upon completing a manual search (UI or CLI), the system immediately synchronizes the complete historical database (`/api/records`) into the frontend memory model, guaranteeing that researchers can filter and inspect all accumulated records without losing historical context.

9. Quality Assurance & Automated Test Architecture

The repository enforces robust software quality standards, verified by an automated test suite containing **140 passing tests** executed via `pytest`.

```
tests/
├─ test_traceability.py      # Citation verification & cognitive invariant tests
├─ test_analyzer.py         # Groq AI prompt structure & taxonomy extraction tests
├─ test_aggregator.py       # Statistical cross-tabulation & pattern synthesis tests
├─ test_insight_reporter.py  # Markdown executive summary & JSON report tests
├─ test_evidence_loader.py   # ImmutabilityGuard & SHA-256 fingerprint tests
├─ test_taxonomy_config.py   # 7-dimension taxonomy schema validation tests
├─ test_main_cli.py         # Unified CLI orchestration & argument tests
├─ test_v0_enhancements.py   # Multi-query tracking & contextual comment tests
├─ test_collector.py        # Post harvesting & timestamp normalization tests
├─ test_reddit_client.py    # RSS XML parsing & 429 adaptive backoff tests
├─ test_deduplicator.py     # Cross-batch deduplication & index tests
├─ test_structurer.py      # Zero-truncation JSON/CSV serialization tests
├─ test_config_loader.py    # Environment substitution & schema tests
├─ test_edge_cases.py      # Unicode, emoji & long-text boundary tests
├─ test_groq_client.py      # API initialization & health check tests
└─ test_scheduler.py        # Scheduled hourly engine & incremental merge tests
```

Key Verification Invariants

- **End-to-End Traceability Invariant:** Confirms that every claim in `insight_report.json` links back to a valid `record_id` and verified Reddit URL.
- **Zero-Truncation Invariant:** Validates that 100% of characters in long post bodies and comment threads are preserved across both JSON and CSV storage layers.
- **Cognitive Stage Completeness:** Verifies that every analyzed record is mapped to one of the 5 mutually exclusive cognitive breakdown stages.
- **Immutability Invariant:** Ensures that attempting to alter a raw record during AI processing raises an exception and aborts execution.

10. Research Ethics, Privacy & Responsible AI Governance

The research engine adheres strictly to established digital research ethics guidelines:

1. **Exclusively Public Discourse:** The system collects only publicly accessible submissions and top comments from open Reddit communities. No private messages, closed subreddits, or restricted profiles are accessed.
2. **Author Pseudonymization:** The configuration layer includes an anonymization toggle (`privacy.anonymize_authors: true`). When enabled, raw usernames are converted into salted SHA-256 pseudonyms (e.g., `anon_8f3a1b2c`) to protect individual privacy in published reports and dashboard views.
3. **Respectful Crawling Protocols:** Reddit client implementations enforce conservative request pacing and strictly respect standard HTTP rate-limiting headers (`Retry-After` , `x-ratelimit-reset`).
4. **Verifiable User Quotes:** Raw user statements are protected against AI hallucination or synthetic mutation by the `ImmutabilityGuard` , ensuring research citations remain completely authentic.

11. Cloud Deployment & Operational Specifications

The system is configured for cloud deployment on platforms like Render or containerized PaaS environments.

Deployment Configuration (`render.yaml`)

```
services:
  - type: web
    name: googlephotos-vague-memory-engine
    env: python
    buildCommand: "pip install -r requirements.txt"
    startCommand: "python app.py"
    envVars:
      - key: PORT
        value: 8000
      - key: GROQ_API_KEY
        sync: false
```

Operational Environment Variables

- `GROQ_API_KEY` (Required for V1 analysis): Groq Cloud API key for LPU-accelerated inference.
- `REDDIT_CLIENT_ID` (Optional): Reddit developer application ID for authenticated PRAW mode.
- `REDDIT_CLIENT_SECRET` (Optional): Reddit developer application secret.
- `REDDIT_USER_AGENT` (Optional): Custom descriptive User-Agent header for HTTP requests.
- `PORT` (Optional, default: `8000`): Web server listening port for the research dashboard.
- `HOST` (Optional, default: `0.0.0.0`): Web server bind address.

12. Conclusion & Strategic Impact

The Google Photos Vague-Memory Research Engine provides an automated, rigorous, and verifiable platform for understanding why modern photo retrieval systems fail human memory. By pairing structured Reddit evidence harvesting with cognitive science taxonomy and high-speed LLM analysis, the system transforms scattered user complaints into actionable engineering insights and product opportunities for next-generation multimodal photo search engines.